

TCP CHAT – Application de communication réseau en Go

Type : Projet scolaire

Période : 2025

Contexte et objectif

Ce projet consiste à développer une application de chat en terminal utilisant le langage **Go** et le protocole **TCP/IP**.

L'objectif principal était de comprendre le fonctionnement des communications réseau bas niveau ainsi que la mise en place d'une architecture client / serveur.

Ce projet m'a permis d'explorer des concepts essentiels comme :

- les sockets TCP
- la communication réseau en temps réel
- la concurrence en Go
- l'utilisation des goroutines

Problématique

Comment mettre en place une communication réseau simple et fonctionnelle entre deux machines en utilisant le protocole TCP/IP, tout en gérant l'envoi et la réception de messages simultanément ?

Technologies utilisées

- Go (Golang)
- Protocole TCP/IP
- Bibliothèque standard Go (**net**, **bufio**, **os**)
- Terminal / CLI

Fonctionnalités principales

- Communication en temps réel entre un client et un serveur
- Architecture client / serveur simple
- Envoi et réception simultanés des messages
- Gestion de la concurrence grâce aux goroutines
- Interface entièrement en terminal (léger et rapide)

Fonctionnement technique

Serveur

Le serveur :

- écoute sur un port TCP (8080)
- attend une connexion d'un client
- reçoit les messages envoyés
- renvoie les messages au client

Il utilise notamment :

- `net.Listen()` pour ouvrir le port
- `Accept()` pour gérer la connexion
- des **goroutines** pour gérer la réception en parallèle de l'envoi

Client

Le client :

- se connecte à l'adresse IP du serveur
- envoie les messages tapés dans le terminal
- reçoit et affiche les messages du serveur

Il utilise :

- `net.Dial()` pour établir la connexion
- `bufio.NewReader()` pour la réception
- `bufio.Scanner` pour la lecture clavier

Difficultés rencontrées

La principale difficulté a été la compréhension du fonctionnement des communications TCP en Go, notamment :

- la gestion des connexions réseau
- la synchronisation entre envoi et réception
- l'utilisation correcte des goroutines sans blocage

Une autre difficulté a été la gestion de l'adresse IP du serveur en environnement local.

Solutions apportées

Pour résoudre ces problèmes :

- j'ai étudié plus en détail le fonctionnement du package `net`

- j'ai utilisé des goroutines pour séparer réception et envoi
- j'ai simplifié l'architecture pour me concentrer sur le fonctionnement TCP de base
- j'ai testé le projet en local puis sur réseau

Résultats et bilan

Le projet aboutit à une application fonctionnelle de chat en ligne de commande permettant une communication simple entre deux machines.

Ce projet m'a permis de :

- comprendre le fonctionnement du protocole TCP/IP
- apprendre la gestion de la concurrence en Go
- améliorer ma logique de programmation réseau
- manipuler des concepts bas niveau liés aux communications

Améliorations possibles

- Gestion de plusieurs clients simultanés
- Ajout de pseudos utilisateurs
- Historique des messages
- Interface graphique (GUI)
- Système de commandes (/quit, /help, etc.)
- Sécurisation des échanges (chiffrement TLS)

Conclusion

Ce projet constitue une première approche concrète des communications réseau en Go.

Il m'a permis de mieux comprendre le fonctionnement des échanges client / serveur et d'acquérir des bases solides en programmation concurrente et réseau.

Auteur

Enzo Courvalet